[Newsletter](#)[HOME](#) > [PROGRAMMING](#)

How to Build and Consume an API in Next.js

Go end-to-end with this tutorial covering every part of a simple API's journey.

BY [MARY GATHONI](#) PUBLISHED OCT 19, 2022



Readers like you help support MUO. When you make a purchase using links on our site, we may earn an affiliate commission. [Read More.](#)

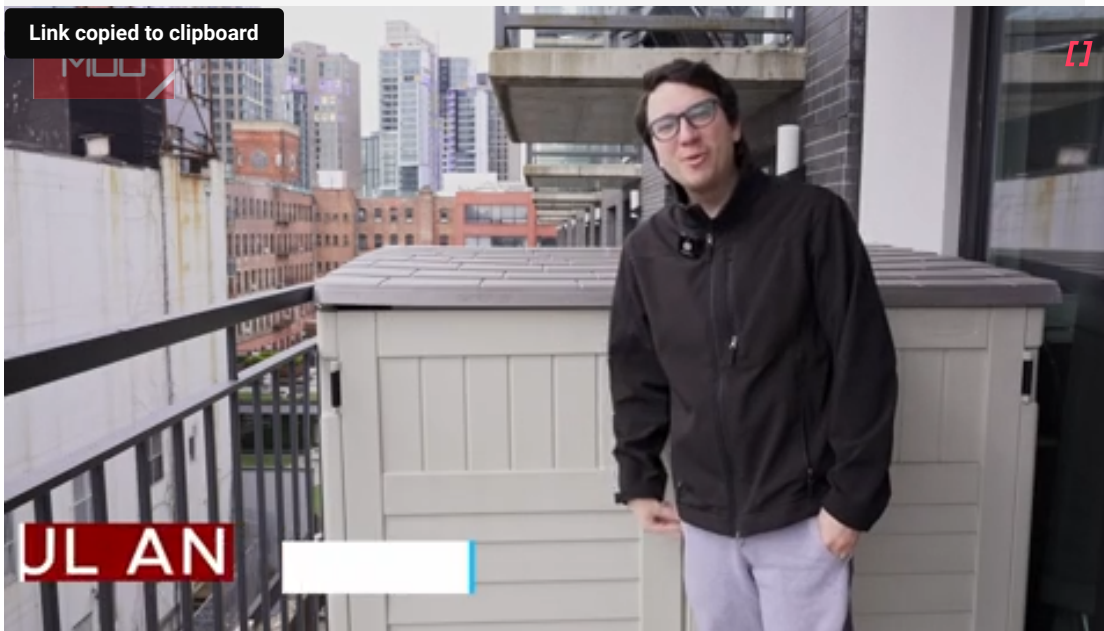
Gone are the days when you had to create a separate backend for your website. With Next.js file-based API routing, you can make your life easier by writing your API inside a Next.js project.

Next.js is a React meta-framework with features that simplify the process of building production-ready web apps. You will see how to build a REST API in Next.js and consume the data from that API on a Next.js page.



The easy and efficient way to display products, streamline sales, and increase revenue.

[LEARN MORE >](#)



Create a Next.js Project Using create-next-app

You can create a new Next.js project using the create-next-app CLI tool. It installs the necessary packages and files to get you started with building a Next.js application.

Run this command in a terminal to create a new Next.js folder called api-routes. You may receive a prompt to install create-next-app.

```
npx create-next-app api-routes
```

When the command finishes, open the api-routes folder to start creating the API routes.

API Routing in Next.js

API routes run on the server and have many uses like saving user data to a database or fetching data from an API without raising the CORS policy error.

In Next.js, you must create API routes inside the /pages/api folder. Next.js generates API endpoints for each of the files in this folder. If you add user.js to /pages/api, Next.js will create an endpoint at http://localhost:3000/api/user.

A basic Next.js API route has the following syntax.

```
Link copied to clipboardion handler(req, res) {  
  res.status(200).json({ name: 'John Doe' })  
}
```

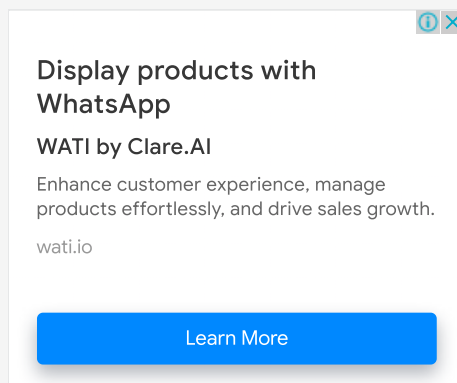
You must export the handler function for it to work.

Creating API Routes

Create a new file called `todo.js` in the `/pages/api` folder to add an API route for todo items.

Mocking the Todo Database

To get the todos, you must create a GET endpoint. For simplicity. This tutorial uses an array of todo items instead of a database but feel free to use a database like MongoDB or MySQL.



Create the todo items in `todo.js` in the root folder of your application then add the following data.

```
export const todos = [  
  {  
    id: 1,  
    todo: "Do something nice for someone I care about",  
    completed: true,  
    userId: 26,  
  },  
  {  
    id: 2,  
    todo: "Memorize the fifty states and their capitals",  
    completed: false,  
    userId: 48,  
  },  
  // other todos  
];
```

These todo items are from the DummyJSON website, a REST API for mock data. You can find the exact data from this DummyJSON todos endpoint.

Next, create the API route in `/pages/api/todos.js` and add the handler function.

Link copied to clipboard

```
import { todos } from "../../todo";

export function handler(req, res) {
  const { method } = req;

  switch (method) {
    case "GET":
      res.status(200).json(todos);
      break;
    case "POST":
      const { todo, completed } = req.body;
      todos.push({
        id: todos.length + 1,
        todo,
        completed,
      });
      res.status(200).json(todos);
      break;
    default:
      res.setHeader("Allow", ["GET", "POST"]);
      res.status(405).end(`Method ${method} Not Allowed`);
      break;
  }
}
```

This route handles the GET and POST endpoints. It returns all the todos for the GET request and adds a todo item to the todo database for a POST request. For other methods, the handler returns an error.

Consuming API Routes in the Frontend

You have created an API endpoint that returns a JSON object containing an array of todos.

To consume the API, create a function called `fetchTodos` that retrieves data from the API endpoint. The function uses the `fetch` method but you can also [use Axios to make API requests](#). Then call this function when you click a button.

```
import Head from "next/head";
import { useState } from "react";

export default function Home() {
  const [todos, settodos] = useState([]);

  const fetchTodos = async () => {
    const response = await fetch("/api/todos");
    const data = await response.json();
    settodos(data);
  };

  return (
    <div className={styles.container}>
```

```

<Head>
  <title>e Next App</title>
  <meta name="description" content="Generated by create next app" />
  <link rel="icon" href="/favicon.ico" />
</Head>
<main>
  <button onClick={fetchTodos}>Get todos</button>
  <ul>
    {todos.map((todo) => {
      return (
        <li
          style={{ color: `${todo.completed ? "green" : "red"}` }}
          key={todo.id}
        >
          {todo.todo}:{todo.completed}.
        </li>
      );
    })}
  </ul>
</main>
</div>
);
}

```

The list in this snippet displays the todo items when they are fetched.

For the POST endpoint, create a new function called `saveTodo` that makes a POST request to the API. The fetch request stores the new todo item in the body and returns all the todo items including the new one. The `saveTodo` function then stores them in the `todos` state.

Ad



Cooler Master MasterBox Pro 5 ARGB ATX Mid-Tower with Adaptable Layout E-A...

★★★★★ 6,464

\$109⁹⁹ ✓prime

```

const saveTodo = async () => {
  const response = await fetch("/api/todos", {
    method: "POST",
    body: JSON.stringify(newTodo),
    headers: {
      "Content-Type": "application/json",
    },
  });

  const data = await response.json();

```

```
setttodos(data);
```

[Link copied to clipboard](#)

Then, create a form with a text input bar to receive the new todo item. The submit handler function of this form will call the saveTodo function.

```
import Head from "next/head";
import { useReducer, useState } from "react";
import styles from "../styles/Home.module.css";

export default function Home() {
  const [todos, setttodos] = useState([]);

  const [newTodo, setnewTodo] = useState({
    todo: "",
    completed: false,
  });

  const fetchTodos = async () => {
    // fetchTodos
  };

  const saveTodo = async (e) => {
    const response = await fetch("/api/todos", {
      method: "POST",
      body: JSON.stringify(newTodo),
      headers: {
        "Content-Type": "application/json",
      },
    });
  };

  const data = await response.json();
  setttodos(data);
};

const handleChange = (e) => {
  setnewTodo({
    todo: e.target.value,
  });
};

const handleSubmit = (e) => {
  e.preventDefault();
  saveTodo();
  setnewTodo({
    todo: '',
  });
};

return (
  <div className={styles.container}>
    <Head>
      <title>Create Next App</title>
      <meta name="description" content="Generated by create next app" />
      <link rel="icon" href="/favicon.ico" />
    </Head>
    <main>
```

```
// Fetches the todo items when clicked
Link copied to clipboard click={fetchTodos}>Get todos</button>

// Saves a new todo item when submitted
<form onSubmit={handleSubmit}>
  <input type="text" onChange={handleChange} value={newTodo.todo} />
</form>
<ul>
  { // list todo items }
</ul>
</main>
</div>
);
}
```

The handler adds a new todo to the database every time a user submits the form. Also, this function updates the todos value using the data received from the API which in turn adds the new todo item to the list.

API Routing Is Just One of Next.js's Strengths

You have seen how you build and use a Next.js API route. Now you can create a full stack application without leaving your Next.js project folder. API routing is one of the many benefits that Next.js provides.

Next.js also offers performance optimizations such as code splitting, lazy loading, and built-in CSS support. If you are building a website that needs to be fast and SEO friendly, you should consider Next.js.



Subscribe to our newsletter

 Comments



RELATED TOPICS

PROGRAMMING

REACT

PROGRAMMING

JAVASCRIPT

API

ABOUT THE AUTHOR

Mary Gathoni

(103 Articles Published)



Mary is a staff writer at MUO based in Nairobi. She has a B.Sc in Applied Physics and Computer Science but enjoys working in tech more. She has been coding and writing technical articles since 2020.